Studio 07 - Robust Regression Simulation Analysis

Groups: Work in teams of 2–4.

1. Objective

Investigate how error distributions, dimensionality, and predictor correlation affect linear regression.

1. Does the correctly specified maximum likelihood estimator (MLE) always perform best?
2. Do poor coefficient estimates necessarily produce poor predictions?
   Choose scenarios, analyze the supplied simulation's output, and create figures.

2. Experimental setup

Consider the linear model

$$y = X\beta + \varepsilon, \qquad X_i \overset{\text{iid}}{\sim} N_p(0, \Sigma), \qquad \varepsilon_i \overset{\text{iid}}{\sim} F,$$

with errors independent of the predictors. Use $n = 200$, $\beta_1 = \cdots = \beta_p = 1/\sqrt{p}$, and $\Sigma_{jk} = \rho^{|j-k|}$.

Use two error distributions, both with mean zero and variance one:

- **Normal:** $N(0, 1)$.
- **Laplace:** location zero and **scale** $1/\sqrt{2}$. In NumPy: `rng.laplace(scale=1/np.sqrt(2), size=n)`.

Run the following experiments:

Experiment	Error distribution	Parameter to vary	Hold fixed
A: Comparing estimators	Normal and Laplace, separately	Choose aspect ratios $\gamma = p/n$ spanning small to large values	$\rho = 0$
B: Estimation versus prediction	Normal	Choose correlations ρ spanning zero to near one	$p/n = 0.5$

Choose a manageable grid, using the same aspect ratios for both error distributions in A. Predict the patterns before running; refine after a first plot if useful. Use at least 100 replicates per scenario. The engine generates fresh data each replicate and fits both methods to the same data.

3. Estimators to compare

Method	Implementation	Objectiv
Ordinary least squares (OLS)	`sklearn.linear_model.LinearRegression(fit_intercept=False)`	Minimize squared residuals
Least absolute deviations (LAD)	`sklearn.linear_model.QuantileRegressor(quantile=0.5, alpha=0., fit_intercept=False)`	Minimize absolute residuals

Optional extension: Add a Huber estimator class using the same error distributions.

4. Evaluation metrics

For each scenario and method, estimate

$$\text{Coefficient MSE} = \mathbb{E}\|\widehat{\beta} - \beta\|_2^2, \qquad \text{Prediction MSE} = \mathbb{E}\left[(\widehat{\beta} - \beta)^\top \Sigma (\widehat{\beta} - \beta)\right].$$

Prediction MSE concerns the noiseless signal $X_{\text{new}}^\top \beta$ for independent new predictors with covariance Σ. The engine computes both losses. Average across replicates; estimate each mean's Monte Carlo standard error (MCSE) as the replicate standard deviation divided by the square root of the number of replicates.

5. Implementation guidelines

- **Supplied:** DGP and estimator classes, losses, study runner, random streams, checkpoints, provenance, tests, and Makefile.
- **You should implement:** `make_scenarios()` in `scenarios.py`; `summarize()`, `figure_a()`, and `figure_b()` in `analyze.py`.
- **Scenario contract:** Map stable IDs to settings with `experiment`, `n`, `p`, `rho`, and `error_distribution`. See the README for types and the results schema.
- **Run:** Use `make simulate`, `make analyze`, and `make figures`, or `make all`. Reference results are available while your study runs, so you can work on the figures.

6. Analysis/visualization

Create two publication-quality figures:

1. **Does the correctly specified MLE always win?** Identify the MLE under each error distribution. For A, plot estimated LAD coefficient MSE divided by estimated OLS coefficient

MSE against $\gamma = p/n$, with two subplots (one for normal errors, one for Laplace errors) and a reference line at one. *Use a ratio of means, not a mean of replicate ratios.* Pair methods by scenario and replicate; the supplied `paired_ratio()` helper provides the ratio's MCSE. Describe the ranking and discuss classical MLE efficiency arguments.

2. **Do poor coefficients imply poor predictions?** For experiment B, plot coefficient MSE and prediction MSE against ρ in adjacent panels, showing both methods. Explain any difference between the two metrics in terms of predicting $X\beta$.

   Size each figure roughly 4–6 inches wide and 2.5–4 inches tall. Choose readable labels, clear legends, consistent colors, and sensible scales. Show Monte Carlo uncertainty, for example with error bars of ± 2 MCSE, and consider it when interpreting differences. Question matplotlib's defaults.

7. Deliverables

Submit your code, generated scenario table (`design.csv`) and raw results, both figures, and a short `ANALYSIS.md`: answer each scientific question in 2–4 sentences.